

NF3: Integridad y Calidad de Datos en Entornos Distribuidos



Dimensiones de la Calidad del Dato (Lección 2.1)

La "mala calidad" se manifiesta a través de cinco problemas principales:

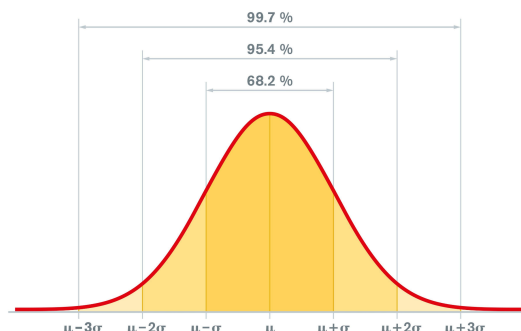
- **Compleitud:** Cuando faltan datos necesarios (ej. un sensor que deja de reportar o un campo obligatorio vacío). 🚫
- **Validez:** El dato no sigue el formato o las reglas definidas (ej. una temperatura de -100°C o un ID con caracteres no permitidos). ❌
- **Consistencia:** Incoherencia entre mediciones (ej. un sensor marca 20°C y un minuto después 200°C). El valor es "válido" por formato, pero ilógico. 📉
- **Unicidad:** Presencia de registros duplicados que pueden falsear las estadísticas (ej. dos entradas idénticas para un mismo evento). 👤
- **Precisión:** El dato es completo y válido, pero no representa la realidad (ej. un GPS que sitúa un autobús en medio del mar). 📍



Detección de Anomalías: Regla de las 3 Desviaciones Estándar (Lección 2.2)

Para detectar valores inconsistentes (**outliers**) de forma objetiva, se utiliza la estadística basada en el histórico de cada sensor.

1. **Cálculo:** Se obtiene la **Media** (μ) y la **Desviación Estándar** (σ) del sensor.
2. **La Regla:** En una distribución normal, el **99.7%** de los datos se encuentran dentro del intervalo:
$$[\mu - 3\sigma, \mu + 3\sigma]$$
3. **Anomalía:** Cualquier dato fuera de ese rango se considera un outlier.
4. **Ventaja:** Es un método **contextual**. El umbral de "normalidad" se adapta automáticamente al comportamiento histórico de cada sensor específico. ⚙️



Integridad en el Data Lake: La Señal **_SUCCESS** (Lección 2.3)

En entornos distribuidos (como Spark), las escrituras de datos pueden fallar a mitad del proceso, dejando carpetas con datos incompletos o corruptos.

- **El Mecanismo:** Cuando un motor de procesamiento finaliza una escritura correctamente, genera automáticamente un fichero vacío llamado **_SUCCESS** dentro de la carpeta de destino. 
- **La Regla de Oro:** Antes de que cualquier proceso lea datos de una carpeta en el Data Lake, debe verificar la existencia de este fichero.
 - **Si existe:** Los datos son íntegros y fiables.
 - **Si NO existe:** Los datos están corruptos o incompletos; el proceso debe detenerse inmediatamente. 

Comparativa de Enfoques de Calidad de Datos

Criterio	 Ad-Hoc (Scripting Manual)	 Frameworks Declarativos	 Plataformas Comerciales
Ejemplos	PySpark (código puro)	Great Expectations, Deequ	Collibra, Informatica DQ
Analogía	Artesano en su taller	Cadena de montaje (QA)	Laboratorio "llave en mano"
Filosofía	Control total y código explícito.	Separación de reglas y ejecución.	Calidad como servicio gobernado.
Definición	Mediante código (<code>df.filter...</code>)	Configuración (<code>expect_...</code>)	Interfaz Gráfica (GUI / Arrastrar)
Resultado	DataFrame con flags o conteos.	Informe estructurado (JSON).	Dashboards y puntuaciones.

Docs	Manual (tienes que crearla).	Automática (Data Docs HTML).	Integrada con linaje de datos.
Integración	Manual en el script/pipeline.	Natural en DAGs (Airflow).	Conectores y APIs complejos.

Análisis de Ventajas y Desventajas

- **Enfoque Ad-Hoc**
 - **Pros:** Flexibilidad ilimitada y sin dependencias extra. 🔒
 - **Contras:** Difícil de mantener, poco reutilizable y sin documentación automática. ⚠️
- **Frameworks Declarativos**
 - **Pros:** Reutilizable, autodocumentado y permite "pruebas unitarias" para datos. ⚙️
 - **Contras:** Curva de aprendizaje para la librería y menos flexible en reglas extremas.
- **Plataformas Comerciales**
 - **Pros:** Fácil para perfiles no técnicos (Data Stewards) y gobernanza centralizada. 📊
 - **Contras:** Coste de licencia muy alto, efecto "caja negra" y dependencia del proveedor (*Vendor Lock-in*).

El Camino de la Madurez (Conclusión)

La gestión de la calidad evoluciona según crece el proyecto:

1. **Nivel Inicial:** Scripts manuales para problemas inmediatos.
2. **Nivel Medio:** Adopción de **frameworks declarativos** para estandarizar y automatizar (estándar de la industria en DataOps). 🏆
3. **Nivel Corporativo:** Inversión en plataformas comerciales para gobernar a gran escala.

Nota: Aprender la lógica "bajo el capó" con Spark (Ad-Hoc) es la base necesaria para dominar después herramientas avanzadas como Great Expectations.